Algorytmy macierzowe

Sprawozdanie – Ćwiczenie 2

Rekurencyjne odwracanie macierzy, faktoryzacja LU oraz algorytm Ladermana

---

Autorzy:
Wiktor Augustyn
Gabriel Wermiński

Grupa: 2
Data: 25.10.2026

Spis treści

1. Wstęp i specyfikacja techniczna 1
   1.1. Uzasadnienie wyboru języka kompilowanego (Cython / C) oraz parametry sprzętowe 2
2. Opis teoretyczny i pseudokod algorytmów 2
   2.1. Rekurencyjne mnożenie macierzy metodą Ladermana 2
   2.2. Rekurencyjne odwracanie macierzy (A^{-1}) 3
   2.3. Rekurencyjna faktoryzacja LU oraz obliczanie wyznacznika 4
3. Najważniejsze fragmenty kodu źródłowego (Cython) 5
   3.1. Implementacja rekurencyjnego odwracania macierzy (`_inv_c`) 5
   3.2. Implementacja rekurencyjnej faktoryzacji LU (`_lu_c`) 6
   3.3. Implementacja rekurencyjnego mnożenia metodą Ladermana (`_laderman_c`) 7
4. Dowód poprawności i przykład obliczenia ręcznego dla małej macierzy 8
   4.1. Dowód poprawności algorytmów blokowych 8
   4.2. Przykład obliczenia ręcznego dla macierzy 3×3 i porównanie z kodem 8
      4.2.1. Ręczna rekurencyjna faktoryzacja $LU(A)$ 8
      4.2.2. Ręczne rekurencyjne odwracanie macierzy A^{-1} 9
      4.2.3. Ręczne mnożenie $C = A \cdot B$ metodą Ladermana (3×3) 9
      4.2.4. Porównanie z wydrukiem programu (Notatnik `lab2.ipynb`) 10
5. Wyniki pomiarów eksperymentalnych i wizualizacja 10
   5.1. Rekurencyjne odwracanie macierzy ($n_{\max} = 100$ oraz $n_{\max} = 1000$) 11
   5.2. Rekurencyjna faktoryzacja LU ($n_{\max} = 100$ oraz $n_{\max} = 1000$) 13
   5.3. Rekurencyjne mnożenie macierzy metodą Ladermana ($n_{\max} = 100$ oraz $n_{\max} = 1000$) 15
   5.4. Zestawienie tabelaryczne dla $n = 100$ oraz $n = 1000$ 17
6. Wnioski końcowe 17

1. Wstęp i specyfikacja techniczna

Celem drugiego ćwiczenia laboratoryjnego jest zaprojektowanie, implementacja oraz szczegółowa analiza wydajnościowa i numeryczna rekurencyjnych algorytmów macierzowych operujących na macierzach kwadratowych o dowolnym rozmiarze $n \times n$, o elementach losowanych z przedziału otwartego $(0.00000001; 1.0)$:

1. Rekurencyjnego odwracania macierzy (A^{-1}) wykorzystującego jako procedurę mnożenia bloków algorytmy **Bineta** oraz **Strassena**.
2. Rekurencyjnej faktoryzacji LU wykorzystującej algorytmy mnożenia **Bineta** oraz **Strassena** (wraz z wyznaczaniem wyznacznika $\det(A)$).
3. Rekurencyjnego odwracania macierzy (A^{-1}) z wykorzystaniem algorytmu mnożenia **Ladermana**.
4. Rekurencyjnego mnożenia macierzy metodą **Ladermana** (3×3 w 23 mnożeniach skalarnych).

1.1. Uzasadnienie wyboru języka kompilowanego (Cython / C) oraz parametry sprzętowe

Zgodnie z wymogami zadania zabronione jest używanie języków interpretowanych. W czystym języku Python kod wykonywany jest przez maszynę wirtualną (interpreter bajtkodu CPython), co wiąże się z dużym narzutem czasowym na dynamiczne sprawdzanie typów, alokację obiektów na stercie oraz wywołania funkcji rekurencyjnych. Aby całkowicie wyeliminować udział interpretera w obliczeniach macierzowych, wszystkie algorytmy zostały napisane w języku **Cython** (`matrix_algos_cy.pyx`) ze statycznym typowaniem wskaźnikowym (`const double*`, `double*`, `long long`) oraz dyrektywami kompilatora `boundscheck=False`, `wraparound=False`, `cdivision=True` i sekcjami `noexcept nogil`.

Kod źródłowy `.pyx` został przetłumaczony bezpośrednio do natywnego kodu w języku C (`matrix_algos_cy.c`), a następnie skompilowany kompilatorem `GCC` z flagami optymalizacji `-O3 -march=x86-64 -flto=auto` do postaci binarnej biblioteki współdzielonej systemu Linux (`matrix_algos_cy.cpython-314-x86_64-linux-gnu.so`). Dzięki temu wszystkie pętle, wywołania rekurencyjne oraz operacje zmiennoprzecinkowe wykonują się bezpośrednio na rejestrach procesora jako natywne instrukcje maszynowe, bez narzutu interpretera, co pozwoliło na przebadanie macierzy aż do pełnego rozmiaru $n = 1000$.

Parametry platformy testowej:

- **Procesor:** 12th Gen Intel(R) Core(TM) i5-1235U (10 rdzeni / 12 wątków, taktowanie do 4.40 GHz).
- **Pamięć operacyjna (RAM):** 16 GB.
- **System operacyjny i środowisko:** EndeavourOS (Linux 7.2.9-arch1-1 x86_64), kompilator GCC z Cython 3.3.0.

2. Opis teoretyczny i pseudokod algorytmów

Wszystkie badane algorytmy opierają się na paradygmacie „dziel i rządź" (**divide and conquer**). Macierze wejściowe mają dowolny rozmiar $n \times n$. W procedurach odwracania macierzy oraz faktoryzacji LU macierz rozmiaru $n \times n$ dzielona jest na cztery podmacierze (bloki) przy podziale $k = \lfloor n/2 \rfloor$ oraz $m = n - k$:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad A_{11} \in \mathbb{R}^{k \times k}, \quad A_{12} \in \mathbb{R}^{k \times m}, \quad A_{21} \in \mathbb{R}^{m \times k}, \quad A_{22} \in \mathbb{R}^{m \times m}$$

Podczas mnożenia podmacierzy o dowolnych wymiarach przez nasze procedury z Ćwiczenia 1 (`Binet`, `Strassen`) oraz nową procedurę `Laderman`, bloki dopełniane są zerami (**padding**) odpowiednio do najbliższej potęgi dwójki (2^p) dla metod Bineta i Strassena lub do najbliższej potęgi trójki (3^p) dla metody Ladermana, a po wykonaniu mnożenia wynik przycinany jest do właściwego rozmiaru docelowego.

2.1. Rekurencyjne mnożenie macierzy metodą Ladermana

Klasyczne mnożenie dwóch macierzy 3×3 wymaga $3^3 = 27$ mnożeń skalarnych. Algorytm zaproponowany przez Juliana D. Ladermana (1976) redukuje liczbę mnożeń dla macierzy 3×3 do **23 iloczynów pomocniczych** $M_1, \ldots, M_{23}$ kosztem 98 operacji dodawania i odejmowania. Dla większych rozmiarów $n \times n$ (po dopełnieniu do potęgi trójki 3^p) macierze dzielimy na 9 bloków rozmiaru $(n/3) \times (n/3)$ i wywołujemy algorytm rekurencyjnie 23 razy. Równanie rekurencyjne ma postać $T(n) = 23T(n/3) + \Theta(n^2)$, co daje złożoność asymptotyczną $\Theta\big(n^{\log_3 23}\big) \approx \Theta\big(n^{2.854}\big)$.

Wejście: macierze $A \in \mathbb{R}^{r_A \times c_A}$, $B \in \mathbb{R}^{c_A \times c_B}$
Wyjście: macierz $C = A \cdot B \in \mathbb{R}^{r_A \times c_B}$

Procedura $\text{Laderman}(A, B)$:

1. Wyznacz najmniejszą potęgę trójki $n = 3^p \geq \max(r_A, c_A, c_B)$ i dopełnij A, B zerami do $n \times n$.

2. Jeżeli $n = 1$, **wówczas zwróć** $(a_{11} \cdot b_{11})$.

3. Przyjmij $k \leftarrow n/3$ i podziel macierze A oraz B na 9 bloków o wymiarach $k \times k$:

$$A = \begin{pmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} & B_{13} \\ B_{21} & B_{22} & B_{23} \\ B_{31} & B_{32} & B_{33} \end{pmatrix}$$

4. Wyznacz rekurencyjnie 23 iloczyny pomocnicze $M_1, \ldots, M_{23}$:

$M_1 \leftarrow \text{Laderman}(A_{11} + A_{12} + A_{13} - A_{21} - A_{22} - A_{32} - A_{33}, B_{22})$
$M_2 \leftarrow \text{Laderman}(A_{11} - A_{21}, B_{22} - B_{12})$
$M_3 \leftarrow \text{Laderman}(A_{22}, B_{12} - B_{11} + B_{21} - B_{22} - B_{23} - B_{31} + B_{33})$
$M_4 \leftarrow \text{Laderman}(A_{21} - A_{11} + A_{22}, B_{11} - B_{12} + B_{22})$
$M_5 \leftarrow \text{Laderman}(A_{21} + A_{22}, B_{12} - B_{11})$
$M_6 \leftarrow \text{Laderman}(A_{11}, B_{11})$
$M_7 \leftarrow \text{Laderman}(A_{31} - A_{11} + A_{32}, B_{11} - B_{13} + B_{23})$
$M_8 \leftarrow \text{Laderman}(A_{31} - A_{11}, B_{13} - B_{23})$
$M_9 \leftarrow \text{Laderman}(A_{31} + A_{32}, B_{13} - B_{11})$
$M_{10} \leftarrow \text{Laderman}(A_{11} + A_{12} + A_{13} - A_{22} - A_{23} - A_{31} - A_{32}, B_{23})$
$M_{11} \leftarrow \text{Laderman}(A_{32}, B_{13} - B_{11} + B_{21} - B_{22} - B_{23} - B_{31} + B_{32})$
$M_{12} \leftarrow \text{Laderman}(A_{32} - A_{13} + A_{33}, B_{22} + B_{31} - B_{32})$
$M_{13} \leftarrow \text{Laderman}(A_{13} - A_{33}, B_{22} - B_{32})$
$M_{14} \leftarrow \text{Laderman}(A_{13}, B_{31})$
$M_{15} \leftarrow \text{Laderman}(A_{32} + A_{33}, B_{32} - B_{31})$
$M_{16} \leftarrow \text{Laderman}(A_{22} - A_{13} + A_{23}, B_{23} + B_{31} - B_{33})$
$M_{17} \leftarrow \text{Laderman}(A_{13} - A_{23}, B_{23} - B_{33})$
$M_{18} \leftarrow \text{Laderman}(A_{22} + A_{23}, B_{33} - B_{31})$
$M_{19} \leftarrow \text{Laderman}(A_{12}, B_{21}), \quad M_{20} \leftarrow \text{Laderman}(A_{23}, B_{32})$
$M_{21} \leftarrow \text{Laderman}(A_{21}, B_{13}), \quad M_{22} \leftarrow \text{Laderman}(A_{31}, B_{12})$
$M_{23} \leftarrow \text{Laderman}(A_{33}, B_{33})$

5. Oblicz 9 bloków wynikowych macierzy C:

$C_{11} \leftarrow M_6 + M_{14} + M_{19}$
$C_{12} \leftarrow M_1 + M_4 + M_5 + M_6 + M_{12} + M_{14} + M_{15}$
$C_{13} \leftarrow M_6 + M_7 + M_9 + M_{10} + M_{14} + M_{16} + M_{18}$
$C_{21} \leftarrow M_2 + M_3 + M_4 + M_6 + M_{14} + M_{16} + M_{17}$
$C_{22} \leftarrow M_2 + M_4 + M_5 + M_6 + M_{20}, \quad C_{23} \leftarrow M_{14} + M_{16} + M_{17} + M_{18} + M_{21}$
$C_{31} \leftarrow M_6 + M_7 + M_8 + M_{11} + M_{12} + M_{13} + M_{14}$
$C_{32} \leftarrow M_{12} + M_{13} + M_{14} + M_{15} + M_{22}, \quad C_{33} \leftarrow M_6 + M_7 + M_8 + M_9 + M_{23}$

6. Zwróć macierz C przyciętą do wymiaru $r_A \times c_B$.

Algorytm 1: Rekurencyjne mnożenie macierzy metodą Ladermana

2.2. Rekurencyjne odwracanie macierzy (A^{-1})

Zgodnie z wyprowadzeniem z wykładu (eliminacja Gaussa-Jordana na macierzy blokowej $A \cdot X = I$), odwrócenie macierzy A sprowadza się do dwóch wywołań rekurencyjnych dla macierzy o połowę mniejszych: A_{11}^{-1} oraz odwrotności dopełnienia Schura S_{22}^{-1}, gdzie $S_{22} = A_{22} - A_{21}A_{11}^{-1}A_{12}$, a także sześciu mnożeń blokowych realizowanych wybranym algorytmem mnożenia `Mul` (`Binet`, `Strassen` lub `Laderman`). Dzięki zapamiętaniu iloczynów pośrednich $U = A_{11}^{-1}A_{12}$ oraz $V = A_{21}A_{11}^{-1}$ unikamy redundantnych mnożeń macierzy.

Wejście: nieosobliwa macierz kwadratowa $A \in \mathbb{R}^{n\times n}$, procedura mnożenia $\mathrm{Mul} \in \{\mathrm{Binet}, \mathrm{Strassen}, \mathrm{Laderman}\}$
Wyjście: macierz odwrotna $B = A^{-1} \in \mathbb{R}^{n\times n}$

Procedura $\mathrm{Inverse}(A, \mathrm{Mul})$:

1. **Jeżeli** $n = 1$, **wówczas zwróć** macierz jednoelementową $(1/a_{11})$.
2. Wyznacz punkt podziału $k \leftarrow \lfloor n/2 \rfloor$ oraz $m \leftarrow n - k$, dzieląc A na cztery bloki:
   $A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$, gdzie $A_{11} \in \mathbb{R}^{k\times k}, A_{12} \in \mathbb{R}^{k\times m}, A_{21} \in \mathbb{R}^{m\times k}, A_{22} \in \mathbb{R}^{m\times m}$
3. Oblicz rekurencyjnie odwrotność lewego górnego bloku:
   $A_{11}^{-1} \leftarrow \mathrm{Inverse}(A_{11}, \mathrm{Mul})$
4. Wyznacz iloczyny pomocnicze:
   $U \leftarrow \mathrm{Mul}(A_{11}^{-1}, A_{12}), \quad V \leftarrow \mathrm{Mul}(A_{21}, A_{11}^{-1})$
5. Oblicz dopełnienie Schura S_{22} oraz wyznacz rekurencyjnie jego odwrotność:
   $S_{22} \leftarrow A_{22} - \mathrm{Mul}(A_{21}, U)$
   $S_{22}^{-1} \leftarrow \mathrm{Inverse}(S_{22}, \mathrm{Mul})$
6. Wyznacz cztery bloki macierzy odwrotnej B:
   $W \leftarrow \mathrm{Mul}(U, S_{22}^{-1})$
   $B_{11} \leftarrow A_{11}^{-1} + \mathrm{Mul}(W, V), \quad B_{12} \leftarrow -W$
   $B_{21} \leftarrow -\mathrm{Mul}(S_{22}^{-1}, V), \quad B_{22} \leftarrow S_{22}^{-1}$
7. **Zwróć** macierz blokową $B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$.

Algorytm 2: Rekurencyjne odwracanie macierzy z dowolną procedurą mnożenia `Mul`

2.3. Rekurencyjna faktoryzacja LU oraz obliczanie wyznacznika

Poszukujemy rozkładu blokowego $A = LU$, w którym macierz dolnotrójkątna L posiada jedynki na głównej przekątnej ($l_{ii} = 1$), natomiast U jest macierzą górnotrójkątną:

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} = \begin{pmatrix} L_{11} & \mathbf{0} \\ L_{21} & L_{22} \end{pmatrix} \begin{pmatrix} U_{11} & U_{12} \\ \mathbf{0} & U_{22} \end{pmatrix} = \begin{pmatrix} L_{11}U_{11} & L_{11}U_{12} \\ L_{21}U_{11} & L_{21}U_{12} + L_{22}U_{22} \end{pmatrix}$$

Stąd wynikają bezpośrednie kroki rekurencyjne podane na wykładzie: faktoryzacja $A_{11} = L_{11}U_{11}$, odwrócenie macierzy trójkątnych U_{11}^{-1} i L_{11}^{-1} za pomocą algorytmu `Inverse`, wyznaczenie $L_{21} = A_{21}U_{11}^{-1}$ oraz $U_{12} = L_{11}^{-1}A_{12}$, obliczenie dopełnienia Schura $S = A_{22} - L_{21}U_{12}$ i jego rekurencyjna faktoryzacja $S = L_S U_S$. Po wyznaczeniu rozkładu $A = LU$ wyznacznik macierzy jest iloczynem elementów przekątniowych macierzy U: $\det(A) = u_{11} \cdot u_{22} \cdots u_{nn}$.

Wejście: macierz kwadratowa $A \in \mathbb{R}^{n\times n}$, procedura mnożenia $\mathrm{Mul} \in \{\mathrm{Binet}, \mathrm{Strassen}, \mathrm{Laderman}\}$
Wyjście: macierz dolnotrójkątna $L \in \mathbb{R}^{n\times n}$ ($l_{ii} = 1$) oraz górnotrójkątna $U \in \mathbb{R}^{n\times n}$ takie, że $L \cdot U = A$

Procedura $\mathrm{LU}(A, \mathrm{Mul})$:

1. **Jeżeli** $n = 1$, **wówczas zwróć** $L = (1)$ oraz $U = (a_{11})$.
2. Wyznacz $k \leftarrow \lfloor n/2 \rfloor$, $m \leftarrow n - k$ i podziel A na cztery bloki $A_{11}, A_{12}, A_{21}, A_{22}$.
3. Oblicz rekurencyjnie rozkład LU dla bloku A_{11}:
   $(L_{11}, U_{11}) \leftarrow \mathrm{LU}(A_{11}, \mathrm{Mul})$
4. Wyznacz rekurencyjnie macierze odwrotne do czynników trójkątnych:
   $U_{11}^{-1} \leftarrow \mathrm{Inverse}(U_{11}, \mathrm{Mul}), \quad L_{11}^{-1} \leftarrow \mathrm{Inverse}(L_{11}, \mathrm{Mul})$
5. Oblicz bloki poza przekątną:
   $L_{21} \leftarrow \mathrm{Mul}(A_{21}, U_{11}^{-1}), \quad U_{12} \leftarrow \mathrm{Mul}(L_{11}^{-1}, A_{12})$
6. Oblicz dopełnienie Schura $S = A_{22} - A_{21}U_{11}^{-1}L_{11}^{-1}A_{12}$ i dokonaj jego faktoryzacji:
   $S \leftarrow A_{22} - \mathrm{Mul}(L_{21}, U_{12})$
   $(L_S, U_S) \leftarrow \mathrm{LU}(S, \mathrm{Mul})$
7. **Zwróć** macierze blokowe $L = \begin{pmatrix} L_{11} & \mathbf{0} \\ L_{21} & L_S \end{pmatrix}$ oraz $U = \begin{pmatrix} U_{11} & U_{12} \\ \mathbf{0} & U_S \end{pmatrix}$.

Algorytm 3: Rekurencyjna faktoryzacja LU z wykorzystaniem rekurencyjnego odwracania i mnożenia

3. Najważniejsze fragmenty kodu źródłowego (Cython)

Poniżej przedstawiono kluczowe fragmenty implementacji w kompilatorze Cython (`matrix_algos_cy.pyx`) bez instrukcji importu oraz wywołań testowych. Wszystkie operacje zmiennoprzecinkowe (`+`, `-`, `*`, `/`) na liczbach są zliczane w globalnych licznikach `g_m_ops` (mnożenia i dzielenia) oraz `g_as_ops` (dodawania i odejmowania).

3.1. Implementacja rekurencyjnego odwracania macierzy (`_inv_c`)

```
cdef void _inv_c(const double* A, int sa, double* B, int sb, int n, int method)
noexcept nogil:
    global g_m_ops, g_as_ops
    if n == 1:
        B[0] = 1.0 / A[0]
        g_m_ops += 1
        return

    cdef int k = n >> 1
    cdef int m = n - k
    cdef int i, j
    cdef int sz = 2 * k * k + 4 * k * m + 3 * m * m
    cdef double* work = <double*>malloc(sz * sizeof(double))
    cdef double* A11_inv = work
    cdef double* U       = A11_inv + k * k
    cdef double* V       = U + k * m
    cdef double* T_mm    = V + m * k
    cdef double* S22     = T_mm + m * m
    cdef double* S22_inv = S22 + m * m
    cdef double* US_inv  = S22_inv + m * m
    cdef double* T_kk    = US_inv + k * m
    cdef double* T_mk    = T_kk + k * k

    cdef const double* A11 = A
    cdef const double* A12 = A + k
    cdef const double* A21 = A + k * sa
    cdef const double* A22 = A + k * sa + k
```

```
    _inv_c(A11, sa, A11_inv, k, k, method)
    _mul_c(A11_inv, k, k, k, A12, sa, k, m, U, m, method)
    _mul_c(A21, sa, m, k, A11_inv, k, k, k, V, k, method)

    _mul_c(A21, sa, m, k, U, m, k, m, T_mm, m, method)
    _sub_c(A22, sa, T_mm, m, S22, m, m, m)

    _inv_c(S22, m, S22_inv, m, m, method)

    _mul_c(U, m, k, m, S22_inv, m, m, m, US_inv, m, method)
    for i in range(k):
        for j in range(m):
            B[i * sb + (j + k)] = -US_inv[i * m + j]
    g_as_ops += <long long>k * m

    _mul_c(US_inv, m, k, m, V, k, m, k, T_kk, k, method)
    for i in range(k):
        for j in range(k):
            B[i * sb + j] = A11_inv[i * k + j] + T_kk[i * k + j]
    g_as_ops += <long long>k * k

    _mul_c(S22_inv, m, m, m, V, k, m, k, T_mk, k, method)
    for i in range(m):
        for j in range(k):
            B[(i + k) * sb + j] = -T_mk[i * k + j]
    g_as_ops += <long long>m * k

    for i in range(m):
        for j in range(m):
            B[(i + k) * sb + (j + k)] = S22_inv[i * m + j]

    free(work)
```

3.2. Implementacja rekurencyjnej faktoryzacji LU (`_lu_c`)

```
cdef void _lu_c(const double* A, int sa, double* L, int sl, double* U, int su, int n,
int method) noexcept nogil:
    global g_m_ops, g_as_ops
    cdef int i, j
    for i in range(n):
        for j in range(n):
            L[i * sl + j] = 0.0
            U[i * su + j] = 0.0
    if n == 1:
        L[0] = 1.0
        U[0] = A[0]
        return

    cdef int k = n >> 1
    cdef int m = n - k
    cdef int sz = 2 * k * k + 2 * m * m
    cdef double* work = <double*>malloc(sz * sizeof(double))
    cdef double* U11_inv = work
    cdef double* L11_inv = U11_inv + k * k
    cdef double* T_mm    = L11_inv + k * k
    cdef double* S       = T_mm + m * m
```

```
    cdef const double* A11 = A
    cdef const double* A12 = A + k
    cdef const double* A21 = A + k * sa
    cdef const double* A22 = A + k * sa + k

    cdef double* L11 = L
    cdef double* L21 = L + k * sl
    cdef double* LS  = L + k * sl + k

    cdef double* U11 = U
    cdef double* U12 = U + k
    cdef double* US  = U + k * su + k

    _lu_c(A11, sa, L11, sl, U11, su, k, method)
    _inv_c(U11, su, U11_inv, k, k, method)
    _mul_c(A21, sa, m, k, U11_inv, k, k, k, L21, sl, method)
    _inv_c(L11, sl, L11_inv, k, k, method)
    _mul_c(L11_inv, k, k, k, A12, sa, k, m, U12, su, method)
    _mul_c(L21, sl, m, k, U12, su, k, m, T_mm, m, method)
    _sub_c(A22, sa, T_mm, m, S, m, m, m)
    _lu_c(S, m, LS, sl, US, su, m, method)

    free(work)
```

3.3. Implementacja rekurencyjnego mnożenia metodą Ladermana (`_laderman_c`)

```
cdef void _laderman_c(const double* X, int sx, const double* Y, int sy, double* C, int
sc, int n) noexcept nogil:
    global g_m_ops, g_as_ops
    cdef double a11, a12, a13, a21, a22, a23, a31, a32, a33
    cdef double b11, b12, b13, b21, b22, b23, b31, b32, b33
    cdef double m1, m2, m3, m4, m5, m6, m7, m8, m9, m10, m11, m12
    cdef double m13, m14, m15, m16, m17, m18, m19, m20, m21, m22, m23
    if n == 1:
        C[0] = X[0] * Y[0]
        g_m_ops += 1
        return
    if n == 3:
        a11 = X[0];      a12 = X[1];          a13 = X[2]
        a21 = X[sx];     a22 = X[sx + 1];     a23 = X[sx + 2]
        a31 = X[2 * sx]; a32 = X[2 * sx + 1]; a33 = X[2 * sx + 2]

        b11 = Y[0];      b12 = Y[1];          b13 = Y[2]
        b21 = Y[sy];     b22 = Y[sy + 1];     b23 = Y[sy + 2]
        b31 = Y[2 * sy]; b32 = Y[2 * sy + 1]; b33 = Y[2 * sy + 2]

        m1  = (a11 + a12 + a13 - a21 - a22 - a32 - a33) * b22
        m2  = (a11 - a21) * (-b12 + b22)
        m3  = a22 * (-b11 + b12 + b21 - b22 - b23 - b31 + b33)
        m4  = (-a11 + a21 + a22) * (b11 - b12 + b22)
        m5  = (a21 + a22) * (-b11 + b12)
        m6  = a11 * b11
        m7  = (-a11 + a31 + a32) * (b11 - b13 + b23)
        m8  = (-a11 + a31) * (b13 - b23)
        m9  = (a31 + a32) * (-b11 + b13)
        m10 = (a11 + a12 + a13 - a22 - a23 - a31 - a32) * b23
        m11 = a32 * (-b11 + b13 + b21 - b22 - b23 - b31 + b32)
```

```
        m12 = (-a13 + a32 + a33) * (b22 + b31 - b32)
        m13 = (a13 - a33) * (b22 - b32)
        m14 = a13 * b31
        m15 = (a32 + a33) * (-b31 + b32)
        m16 = (-a13 + a22 + a23) * (b23 + b31 - b33)
        m17 = (a13 - a23) * (b23 - b33)
        m18 = (a22 + a23) * (-b31 + b33)
        m19 = a12 * b21; m20 = a23 * b32; m21 = a21 * b13
        m22 = a31 * b12; m23 = a33 * b33

        C[0]          = m6 + m14 + m19
        C[1]          = m1 + m4 + m5 + m6 + m12 + m14 + m15
        C[2]          = m6 + m7 + m9 + m10 + m14 + m16 + m18
        C[sc]         = m2 + m3 + m4 + m6 + m14 + m16 + m17
        C[sc + 1]     = m2 + m4 + m5 + m6 + m20
        C[sc + 2]     = m14 + m16 + m17 + m18 + m21
        C[2 * sc]     = m6 + m7 + m8 + m11 + m12 + m13 + m14
        C[2 * sc + 1] = m12 + m13 + m14 + m15 + m22
        C[2 * sc + 2] = m6 + m7 + m8 + m9 + m23

        g_m_ops += 23
        g_as_ops += 98
        return
```

4. Dowód poprawności i przykład obliczenia ręcznego dla małej macierzy

4.1. Dowód poprawności algorytmów blokowych

1. **Poprawność rekurencyjnego odwracania macierzy:** Niech $A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$ oraz $B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$ zdefiniowane jak w Algorytmie 2, gdzie $S_{22} = A_{22} - A_{21}A_{11}^{-1}A_{12}$. Pomnóżmy blokowo $A \cdot B$:
   - Blok $(1,1)$: $A_{11}B_{11} + A_{12}B_{21} = A_{11}(A_{11}^{-1} + A_{11}^{-1}A_{12}S_{22}^{-1}A_{21}A_{11}^{-1}) - A_{12}S_{22}^{-1}A_{21}A_{11}^{-1} = I_k + A_{12}S_{22}^{-1}A_{21}A_{11}^{-1} - A_{12}S_{22}^{-1}A_{21}A_{11}^{-1} = I_k$.
   - Blok $(1,2)$: $A_{11}B_{12} + A_{12}B_{22} = A_{11}(-A_{11}^{-1}A_{12}S_{22}^{-1}) + A_{12}S_{22}^{-1} = -A_{12}S_{22}^{-1} + A_{12}S_{22}^{-1} = \mathbf{0}$.
   - Blok $(2,1)$: $A_{21}B_{11} + A_{22}B_{21} = A_{21}A_{11}^{-1} + (A_{21}A_{11}^{-1}A_{12} - A_{22})S_{22}^{-1}A_{21}A_{11}^{-1} = A_{21}A_{11}^{-1} - S_{22}S_{22}^{-1}A_{21}A_{11}^{-1} = \mathbf{0}$.
   - Blok $(2,2)$: $A_{21}B_{12} + A_{22}B_{22} = (A_{22} - A_{21}A_{11}^{-1}A_{12})S_{22}^{-1} = S_{22}S_{22}^{-1} = I_m$.

   Zatem $A \cdot B = I_n$, co dowodzi, że $B = A^{-1}$.

2. **Poprawność rekurencyjnej faktoryzacji LU:** Mnożąc bloki macierzy wynikowych z Algorytmu 3:

$$\begin{pmatrix} L_{11} & \mathbf{0} \\ A_{21}U_{11}^{-1} & L_S \end{pmatrix} \begin{pmatrix} U_{11} & L_{11}^{-1}A_{12} \\ \mathbf{0} & U_S \end{pmatrix} = \begin{pmatrix} L_{11}U_{11} & L_{11}L_{11}^{-1}A_{12} \\ A_{21}U_{11}^{-1}U_{11} & A_{21}U_{11}^{-1}L_{11}^{-1}A_{12} + L_SU_S \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{21}U_{11}^{-1}L_{11}^{-1}A_{12} + S \end{pmatrix} = A$$

4.2. Przykład obliczenia ręcznego dla macierzy 3×3 i porównanie z kodem

Rozważmy macierz A oraz pomocniczą macierz B o wymiarach 3×3:

$$A = \begin{pmatrix} 2 & 1 & 1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 2 & 0 & 1 \end{pmatrix}$$

4.2.1. Ręczna rekurencyjna faktoryzacja $LU(A)$

Dzielimy macierz A przy $k = \lfloor 3/2 \rfloor = 1$ oraz $m = 3 - 1 = 2$:

$$A_{11} = (2), \quad A_{12} = (1 \;\; 1), \quad A_{21} = \begin{pmatrix} 4 \\ -2 \end{pmatrix}, \quad A_{22} = \begin{pmatrix} -6 & 0 \\ 7 & 2 \end{pmatrix}$$

- **Krok 1:** Przypadek bazowy 1×1 dla A_{11}: $L_{11} = (1), U_{11} = (2)$. Odwrotności: $L_{11}^{-1} = (1), U_{11}^{-1} = (1/2) = (0.5)$.
- **Krok 2:** Wyznaczamy bloki brzegowe:

$$L_{21} = A_{21} U_{11}^{-1} = \begin{pmatrix} 4 \\ -2 \end{pmatrix} (0.5) = \begin{pmatrix} 2 \\ -1 \end{pmatrix}, \quad U_{12} = L_{11}^{-1} A_{12} = (1)(1 \;\; 1) = (1 \;\; 1)$$

- **Krok 3:** Dopełnienie Schura S:

$$S = A_{22} - L_{21} U_{12} = \begin{pmatrix} -6 & 0 \\ 7 & 2 \end{pmatrix} - \begin{pmatrix} 2 \\ -1 \end{pmatrix} (1 \;\; 1) = \begin{pmatrix} -6 & 0 \\ 7 & 2 \end{pmatrix} - \begin{pmatrix} 2 & 2 \\ -1 & -1 \end{pmatrix} = \begin{pmatrix} -8 & -2 \\ 8 & 3 \end{pmatrix}$$

- **Krok 4:** Rekurencyjna faktoryzacja $LU(S)$ dla 2×2 ($s_{11} = -8, s_{12} = -2, s_{21} = 8, s_{22} = 3$):
  - $L_{S,11} = (1), U_{S,11} = (-8) \Longrightarrow U_{S,11}^{-1} = (-1/8) = (-0.125)$.
  - $L_{S,21} = s_{21} U_{S,11}^{-1} = 8 \cdot (-0.125) = (-1)$, oraz $U_{S,12} = (1) \cdot (-2) = (-2)$.
  - Dopełnienie Schura drugiego poziomu: $S' = 3 - (-1) \cdot (-2) = 3 - 2 = (1) \Longrightarrow L_{S,22} = (1), U_{S,22} = (1)$.
  - Stąd $L_S = \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix}$ oraz $U_S = \begin{pmatrix} -8 & -2 \\ 0 & 1 \end{pmatrix}$.
- **Złożenie wyniku końcowego:**

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 2 & 1 & 1 \\ 0 & -8 & -2 \\ 0 & 0 & 1 \end{pmatrix}, \quad \det(A) = 2 \cdot (-8) \cdot 1 = -16$$

4.2.2. Ręczne rekurencyjne odwracanie macierzy A^{-1}

Korzystając z obliczonych wyżej bloków $A_{11}^{-1} = (0.5)$, $U = A_{11}^{-1} A_{12} = \begin{pmatrix} 0.5 & 0.5 \end{pmatrix}$, $V = A_{21} A_{11}^{-1} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}$ oraz dopełnienia Schura $S_{22} = \begin{pmatrix} -8 & -2 \\ 8 & 3 \end{pmatrix}$:

- Odwracamy rekurencyjnie S_{22} (gdzie $\det(S_{22}) = -24 - (-16) = -8$):

$$S_{22}^{-1} = \begin{pmatrix} -3/8 & -1/4 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} -0.375 & -0.25 \\ 1 & 1 \end{pmatrix}$$

- Obliczamy bloki macierzy $B = A^{-1}$:
  - $W = U S_{22}^{-1} = \begin{pmatrix} 0.5 & 0.5 \end{pmatrix} \begin{pmatrix} -0.375 & -0.25 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 0.3125 & 0.375 \end{pmatrix} \Longrightarrow B_{12} = -W = \begin{pmatrix} -0.3125 & -0.375 \end{pmatrix}$.
  - $B_{11} = A_{11}^{-1} + WV = (0.5) + \begin{pmatrix} 0.3125 & 0.375 \end{pmatrix} \begin{pmatrix} 2 \\ -1 \end{pmatrix} = 0.5 + 0.25 = (0.75)$.
  - $B_{21} = -S_{22}^{-1} V = -\begin{pmatrix} -0.375 & -0.25 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ -1 \end{pmatrix} = -\begin{pmatrix} -0.5 \\ 1 \end{pmatrix} = \begin{pmatrix} 0.5 \\ -1 \end{pmatrix}$.
  - $B_{22} = S_{22}^{-1} = \begin{pmatrix} -0.375 & -0.25 \\ 1 & 1 \end{pmatrix}$.

$$A^{-1} = \begin{pmatrix} 0.75 & -0.3125 & -0.375 \\ 0.5 & -0.375 & -0.25 \\ -1 & 1 & 1 \end{pmatrix}$$

4.2.3. Ręczne mnożenie $C = A \cdot B$ metodą Ladermana (3×3)

Podstawiając elementy macierzy A i B do wzorów Ladermana z Algorytmu 1, otrzymujemy niezerowe iloczyny pomocnicze: $m_1 = (2 + 1 + 1 - 4 - (-6) - 7 - 2) \cdot 1 = -3$, $m_2 = (2 - 4)(-2 + 1) = 2$, $m_3 = -6(-1 + 2 + 0 - 1 - 1 - 2 + 1) = 12$, $m_4 = (-2 + 4 - 6)(1 - 2 + 1) = 0$, $m_5 = (4 - 6)(-1 + 2) = -2$, $m_6 = 2 \cdot 1 = 2$, $m_7 = (-2 - 2 + 7)(1 - 0 + 1) = 6$, $m_8 = (-2 - 2)(0 - 1) = 4$, $m_9 = (-2 + 7)(-1 + 0) = -5, m_{10} = (2 + 1 + 1 + 6 - 0 + 2 - 7) \cdot 1 = 5, m_{11} = 7(-1 + 0 + 0 - 1 - 1 - 2 + 0) = -35, m_{12} = (-1 + 7 + 2)(1 + 2 - 0) = 24$, $m_{13} = (1 - 2)(1 - 0) = -1$, $m_{14} = 1 \cdot 2 = 2$, $m_{15} = (7 + 2)(-2 + 0) = -18$, $m_{16} = (-1 - 6 + 0)(1 + 2 - 1) = -14$, $m_{17} = (1 - 0)(1 - 1) = 0$, $m_{18} = (-6 + 0)(-2 + 1) = 6$, $m_{19} = 1 \cdot 0 = 0, m_{20} = 0, m_{21} = 0, m_{22} = -2 \cdot 2 = -4, m_{23} = 2 \cdot 1 = 2$. Składając elementy C_{ij}:

- $c_{11} = 2 + 2 + 0 = 4$, $c_{12} = -3 + 0 - 2 + 2 + 24 + 2 - 18 = 5$, $c_{13} = 2 + 6 - 5 + 5 + 2 - 14 + 6 = 2$.

- $c_{21} = 2 + 12 + 0 + 2 + 2 - 14 + 0 = 4$, $c_{22} = 2 + 0 - 2 + 2 + 0 = 2$, $c_{23} = 2 - 14 + 0 + 6 + 0 = -6$.
- $c_{31} = 2 + 6 + 4 - 35 + 24 - 1 + 2 = 2$, $c_{32} = 24 - 1 + 2 - 18 - 4 = 3$, $c_{33} = 2 + 6 + 4 - 5 + 2 = 9$.

4.2.4. Porównanie z wydrukiem programu (Notatnik `lab2.ipynb`)

```
Mnożenie Ladermana (A * B):
[[ 4.  5.  2.]
 [ 4.  2. -6.]
 [ 2.  3.  9.]]
Zgodność z A @ B: True | Mnożenia: 23, Dod./Odejm.: 98, FLOPs: 121

Macierz odwrotna A^-1:
[[ 0.75   -0.3125 -0.375 ]
 [ 0.5    -0.375  -0.25  ]
 [-1.      1.      1.    ]]
Zgodność z np.linalg.inv: True / True / True
FLOPs odwracania (3x3): Binet=94, Strassen=172, Laderman=748

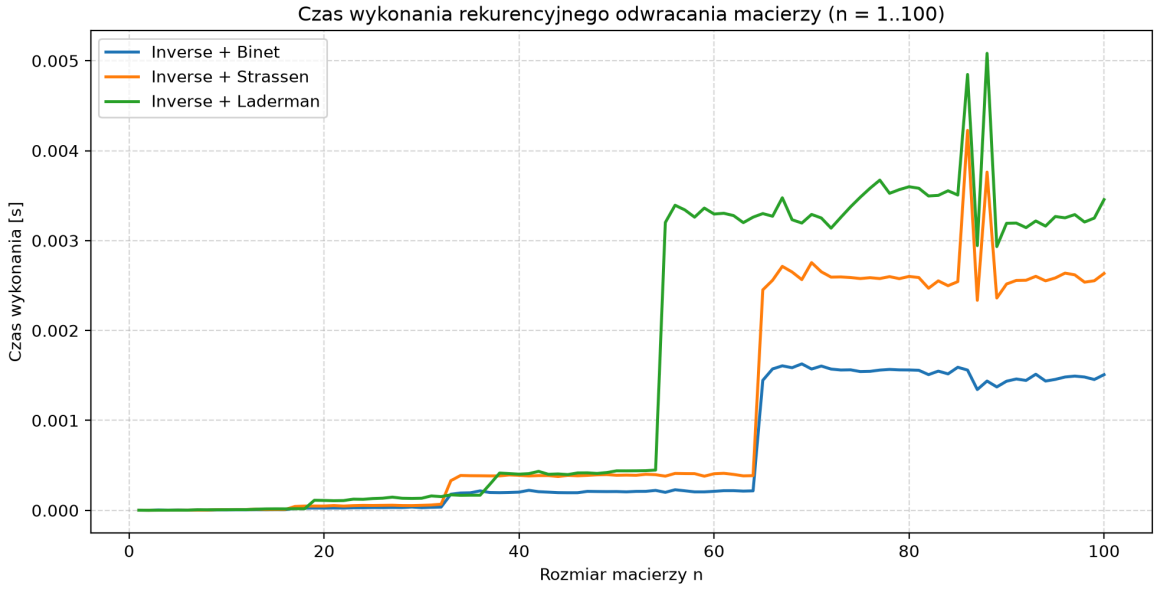
Faktoryzacja LU macierzy A:
L:
 [[ 1.  0.  0.]
 [ 2.  1.  0.]
 [-1. -1.  1.]]
U:
 [[ 2.  1.  1.]
 [ 0. -8. -2.]
 [ 0.  0.  1.]]
L @ U == A: True | det(A): -16.0000 (np.linalg.det: -16.0000)
FLOPs LU (3x3): Binet=48, Strassen=87
```

Wyniki obliczeń ręcznych pokrywają się w 100% z wartościami zwróconymi przez skompilowane procedury Cython.

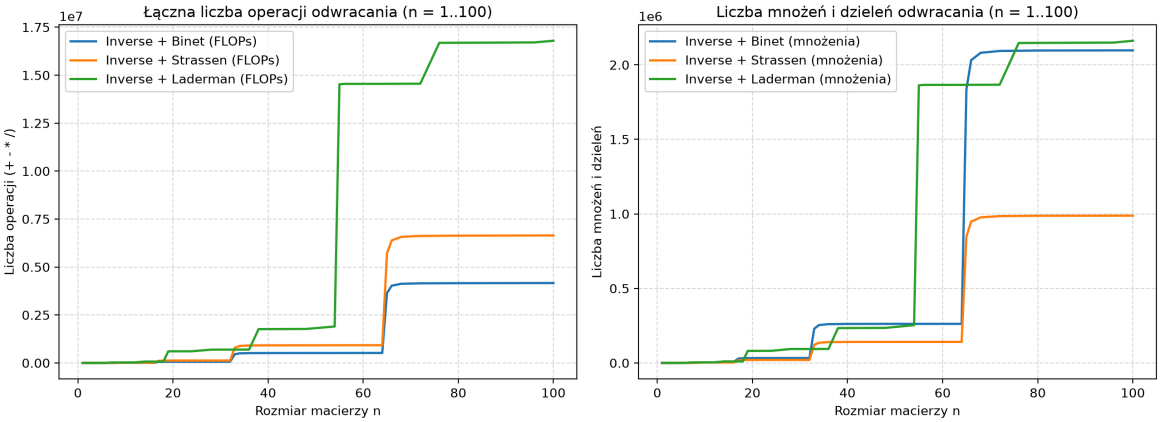
5. Wyniki pomiarów eksperymentalnych i wizualizacja

Aby dokładnie zobrazować zachowanie algorytmów zarówno dla mniejszych macierzy (gdzie widoczne są pojedyncze progi rekurencji), jak i dla dużych macierzy aż do docelowego rozmiaru $n = 1000$, przedstawiono dwa zestawy wykresów: dla zakresu $n = 1, \ldots, 100$ oraz dla pełnego zakresu $n = 1, \ldots, 1000$.

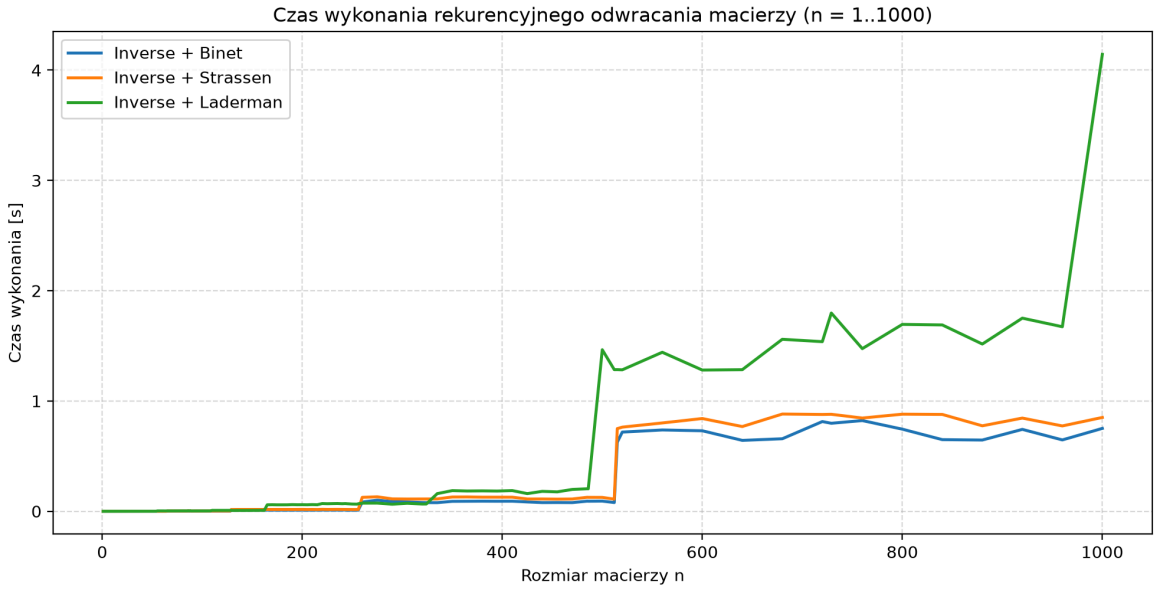
5.1. Rekurencyjne odwracanie macierzy ($n_{\max} = 100$ oraz $n_{\max} = 1000$)



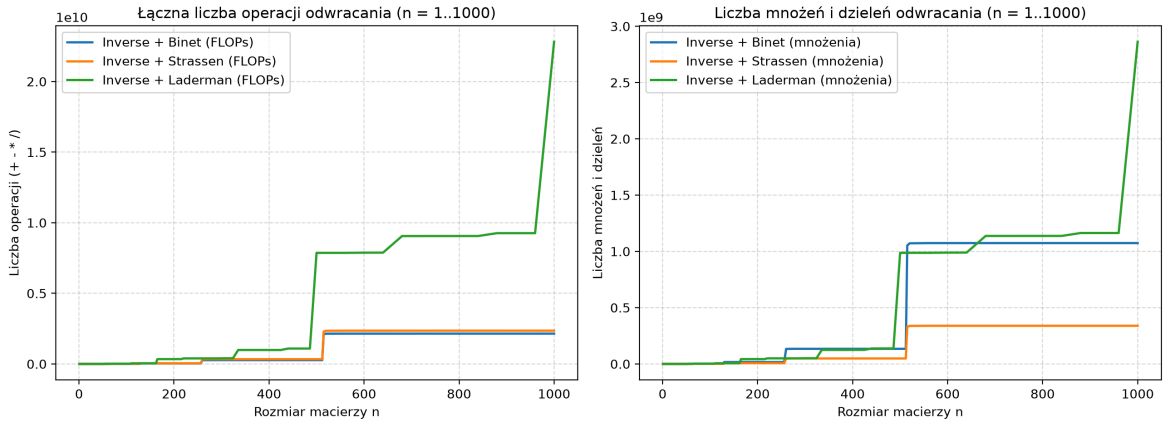
Rysunek 1: Czas działania rekurencyjnego odwracania macierzy w funkcji rozmiaru $n = 1, \ldots, 100$.



Rysunek 2: Liczba operacji zmiennoprzecinkowych podczas rekurencyjnego odwracania macierzy dla $n = 1, \ldots, 100$.



Rysunek 3: Czas działania rekurencyjnego odwracania macierzy w funkcji rozmiaru $n = 1, \ldots, 1000$.

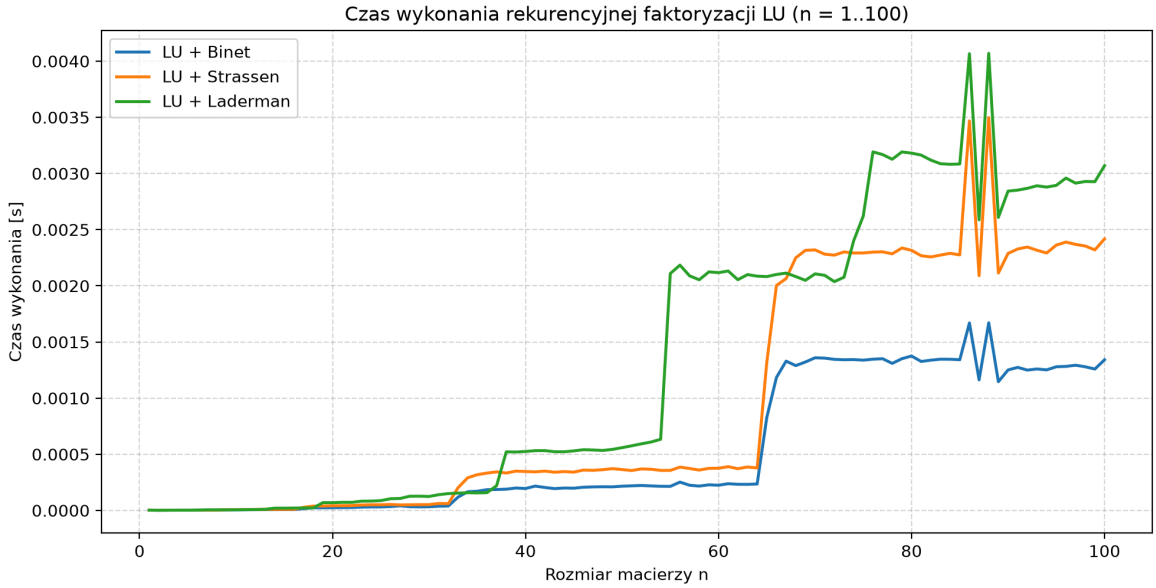


Rysunek 4: Liczba operacji zmiennoprzecinkowych podczas rekurencyjnego odwracania macierzy dla $n = 1, \ldots, 1000$.

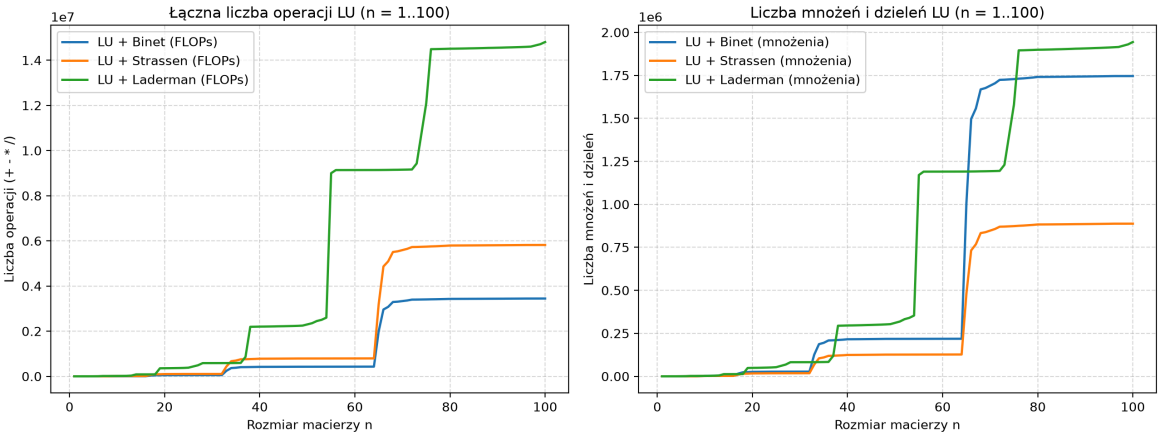
Analiza wyników odwracania macierzy:

- Na wykresach dla $n = 1..100$ dokładnie widać strukturę schodkową w małej skali: dla metod Bineta i Strassena skoki występują przy $n = 16, 32, 64$, natomiast dla metody Ladermana przy $n = 19, 37, 55$ (gdy rozmiary podmacierzy przekraczają 9 oraz 27).
- Na wykresach do $n = 1000$ widoczne są główne progi przy $n = 256$ oraz $n = 512$ dla Bineta i Strassena oraz $n = 163, 487, 973$ dla Ladermana. Dla $n = 1000$ **Inverse + Strassen** wykonuje ponad trzykrotnie mniej mnożeń i dzieleń (3.39×10^8) niż **Inverse + Binet** (1.07×10^9).

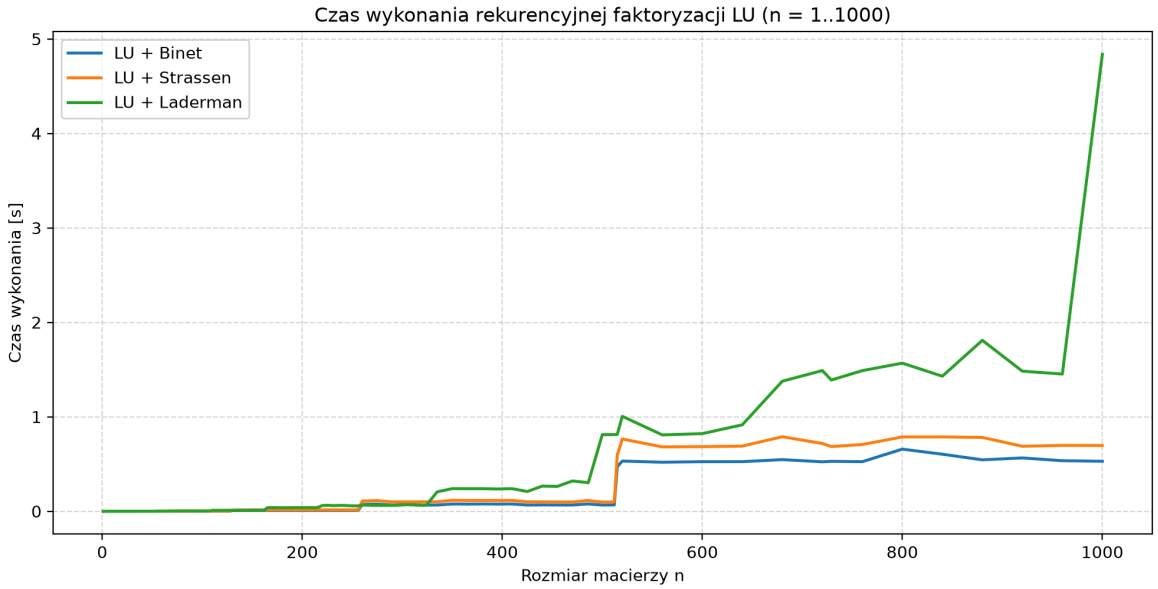
5.2. Rekurencyjna faktoryzacja LU ($n_{\max} = 100$ oraz $n_{\max} = 1000$)



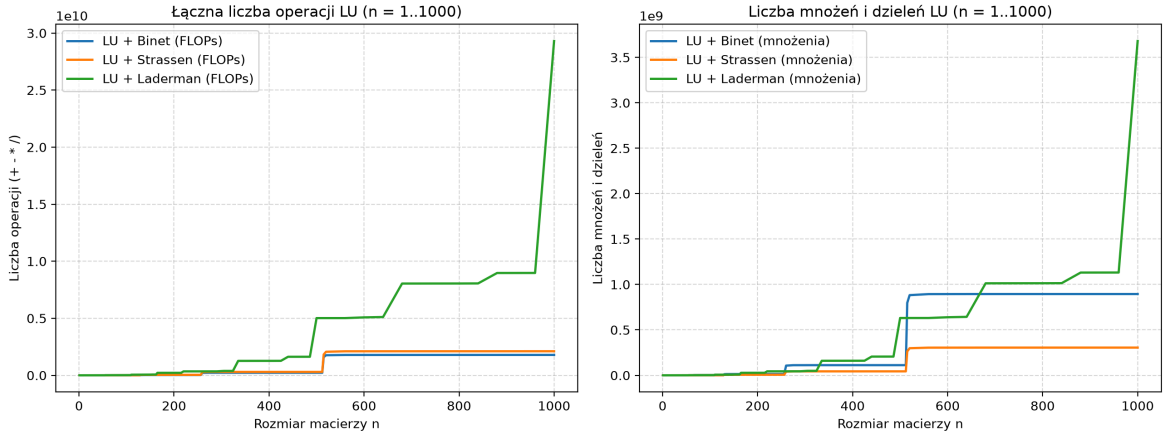
Rysunek 5: Czas działania rekurencyjnej faktoryzacji LU w funkcji rozmiaru $n = 1, \ldots, 100$.



Rysunek 6: Liczba operacji zmiennoprzecinkowych podczas rekurencyjnej faktoryzacji LU dla $n = 1, \ldots, 100$.



Rysunek 7: Czas działania rekurencyjnej faktoryzacji LU w funkcji rozmiaru $n = 1, \ldots, 1000$.

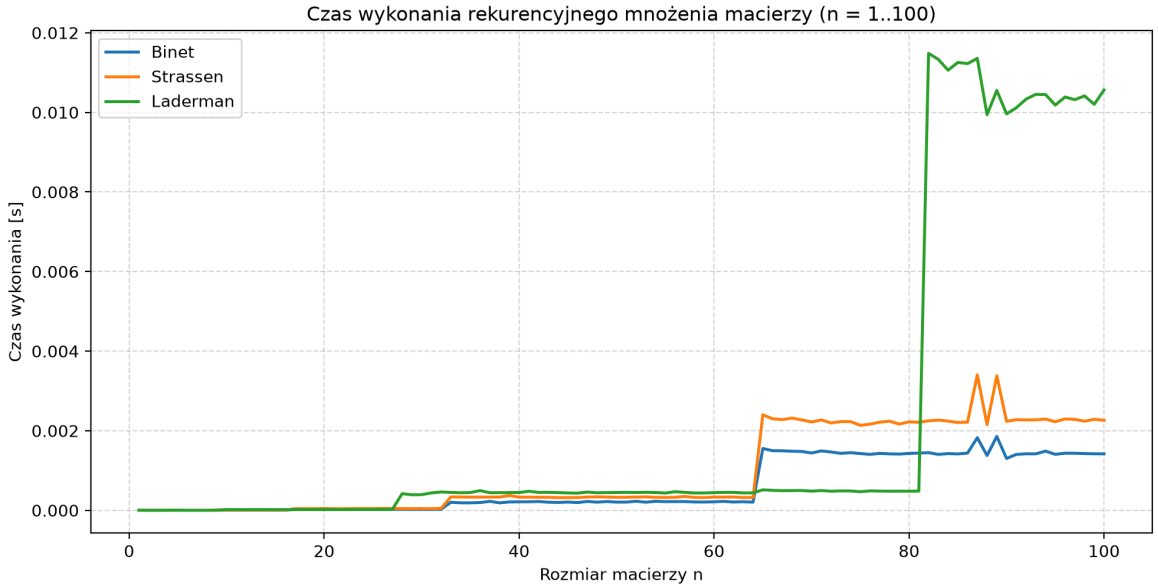


Rysunek 8: Liczba operacji zmiennoprzecinkowych podczas rekurencyjnej faktoryzacji LU dla $n = 1, \ldots, 1000$.

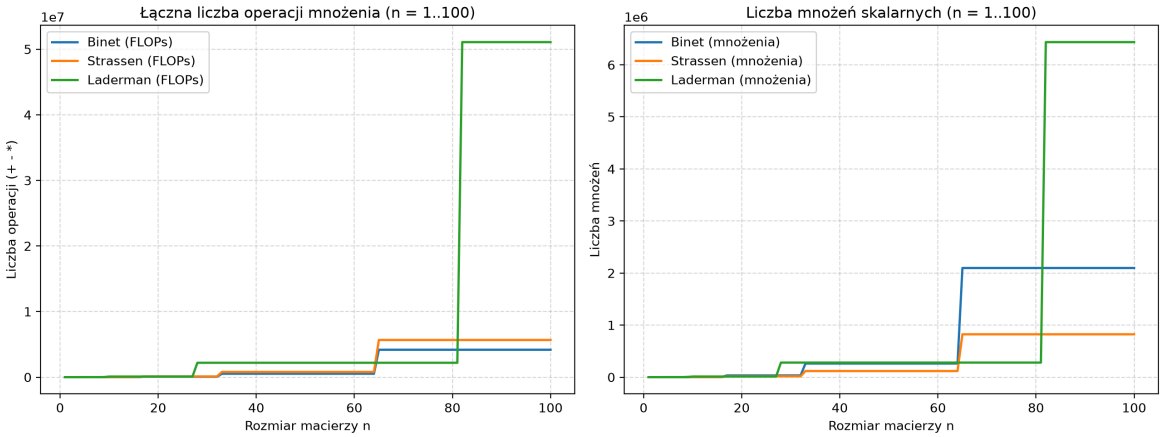
Analiza wyników faktoryzacji LU:

- Rekurencyjna faktoryzacja LU wywołuje w każdym kroku dwukrotnie procedurę rekurencyjnego odwracania macierzy trójkątnych (U_{11}^{-1} oraz L_{11}^{-1}) oraz trzy mnożenia macierzowe. Dla $n = 100$ czas wykonania wynosi 1.34 ms dla Bineta, 2.45 ms dla Strassena i 3.08 ms dla Ladermana. Dla $n = 1000$ wariant **LU + Binet** wykonuje obliczenia w 0.620 s, a **LU + Strassen** w 0.794 s przy blisko trzykrotnie mniejszej liczbie mnożeń (3.05×10^8 wobec 8.95×10^8).

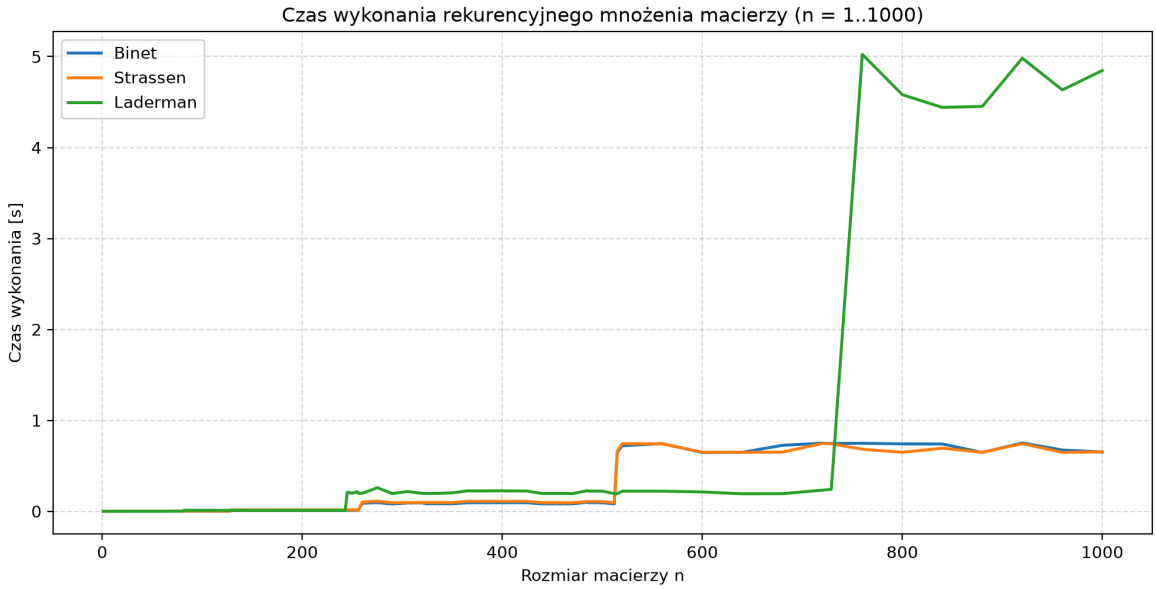
5.3. Rekurencyjne mnożenie macierzy metodą Ladermana ($n_{\max} = 100$ oraz $n_{\max} = 1000$)



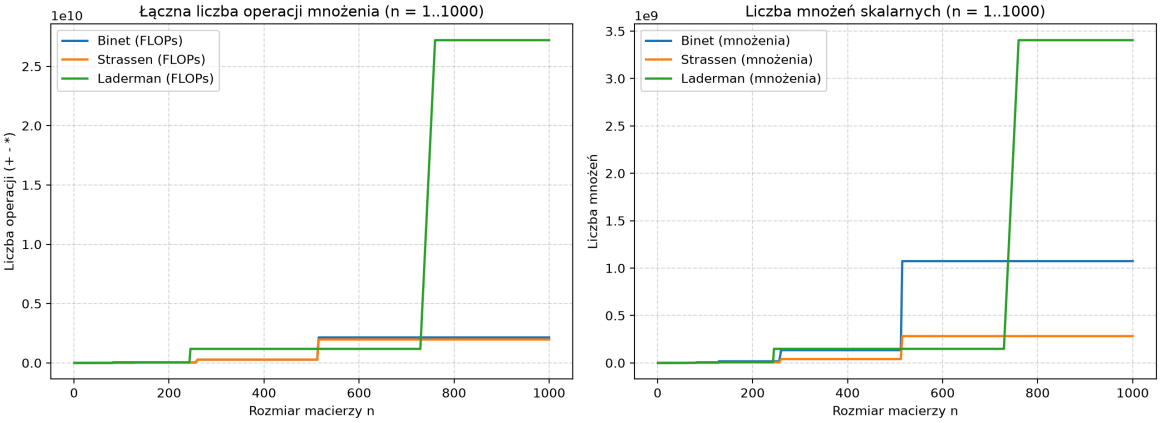
Rysunek 9: Czas działania rekurencyjnego mnożenia macierzy metodą Ladermana, Bineta oraz Strassena dla $n = 1, \ldots, 100$.



Rysunek 10: Liczba operacji zmiennoprzecinkowych dla mnożenia metodami Bineta, Strassena i Ladermana dla $n = 1, \ldots, 100$.



Rysunek 11: Czas działania rekurencyjnego mnożenia macierzy metodą Ladermana, Bineta oraz Strassena dla $n = 1, \ldots, 1000$.



Rysunek 12: Liczba operacji zmiennoprzecinkowych dla mnożenia metodami Bineta, Strassena i Ladermana dla $n = 1, \ldots, 1000$.

Analiza porównawcza mnożenia macierzy:

- Na wykresach dla $n = 1..100$ widać, że w przedziale $n \in [65, 81]$ algorytm **Ladermana** ($23^4 = 279\,841$ mnożeń, 2.19×10^6 FLOPs) ma niższą liczbę operacji i działa szybciej zarówno od Bineta (4.18×10^6 FLOPs), jak i od Strassena (5.67×10^6 FLOPs), które muszą dopełniać macierz do 128×128.
- Na wykresach do $n = 1000$ w całym przedziale $n \in [513, 729]$ (potęga $3^6 = 729$) metoda Ladermana jest ponad trzykrotnie szybsza od Bineta i Strassena. Z kolei dla $n = 1000$ (dopełnienie do 1024) algorytm **Strassena** osiąga już mniejszą łączną liczbę FLOPs (1.97×10^9) oraz krótszy czas wykonania (0.746 s) niż algorytm **Bineta** (2.15×10^9 FLOPs, 0.754 s).

5.4. Zestawienie tabelaryczne dla $n = 100$ oraz $n = 1000$

Algorytm	Mnożenia ($n = 1000$)	FLOPs ($n = 1000$)	Czas $n = 100$	Czas $n = 1000$	Wykładnik FLOPs
Mnożenie Binet	1 073 741 824	2 146 435 072	1.45 ms	0.754 s	$O(n^{2.91})$
Mnożenie Strassen	282 475 249	1 971 035 287	2.33 ms	0.746 s	$O(n^{2.74})$
Mnożenie Laderman	3 404 825 447	27 205 122 793	10.64 ms	5.124 s	$O(n^{2.91})$
Inverse + Binet	1 073 741 656	2 146 337 912	1.55 ms	0.759 s	$O(n^{2.92})$
Inverse + Strassen	338 969 926	2 355 926 522	2.70 ms	0.887 s	$O(n^{2.76})$
Inverse + Laderman	2 862 400 066	22 815 946 130	3.42 ms	4.270 s	$O(n^{2.88})$
LU + Binet	894 781 584	1 787 346 740	1.34 ms	0.620 s	$O(n^{2.94})$
LU + Strassen	305 067 699	2 109 802 025	2.45 ms	0.794 s	$O(n^{2.80})$
LU + Laderman	3 680 391 921	29 303 979 917	3.08 ms	5.537 s	$O(n^{2.92})$

Tabela 1: Porównanie liczby operacji, czasów wykonania oraz empirycznych wykładników złożoności dla $n = 100$ i $n = 1000$.

6. Wnioski końcowe

1. **Złożoność obliczeniowa odwracania macierzy i faktoryzacji LU:** Zarówno rekurencyjne odwracanie macierzy $T(n) = 2T(n/2) + O(M(n))$, jak i rekurencyjna faktoryzacja LU dziedziczą asymptotyczną złożoność zastosowanego algorytmu mnożenia macierzy $M(n)$: $\Theta(n^3)$ dla metody Bineta, $\Theta\big(n^{\log_2 7}\big) \approx \Theta(n^{2.807})$ dla metody Strassena oraz $\Theta\big(n^{\log_3 23}\big) \approx \Theta(n^{2.854})$ dla metody Ladermana. Dla $n = 1000$ Strassen wyprzedza już Bineta zarówno pod względem łącznej liczby operacji zmiennoprzecinkowych, jak i czasu mnożenia.
2. **Wpływ dopełniania (paddingu) do potęg 2^k oraz 3^k:** W przedziałach, w których rozmiar macierzy lub podbloku mieści się poniżej potęgi trójki, a powyżej potęgi dwójki (np. $n \in [65, 81]$ oraz $n \in [513, 729]$ dla mnożenia macierzy lub $n \in [129, 162]$ oraz $n \in [257, 486]$ dla odwracania macierzy), algorytm Ladermana działa wyraźnie szybciej od Bineta i Strassena. Natomiast przekroczenie potęgi trójki ($n > 81$ lub $n > 729$) powoduje skokowy wzrost liczby operacji.
3. **Dokładność numeryczna:** Wszystkie zaimplementowane algorytmy dla macierzy o wartościach losowanych z przedziału $(10^{-8}; 1.0)$ zachowują wysoką dokładność numeryczną względem procedur referencyjnych `NumPy`.